

Binary Information

1. Binary Name = Plain sight
2. Language = C/C++
3. Arch = x86x64
4. Platform = Unix/Linux etc.

```
$ file plain_sight
```

plain_sight: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=48e0a7b04cce9f3b0a3460de3ab5faeba2655625, for GNU/Linux 3.2.0, not stripped

Analysis

I have done the analysis of the binary using IDA free 9.2 version in the Linux system.

Static Analysis

Below is the disassembly of the main function of the program, where we call it calls a function.

Main disassembly

```
.text:000000000000129D
.text:000000000000129D
.text:000000000000129D ; Attributes: bp-based frame
.text:000000000000129D ; int __fastcall main(int argc, const char **argv, const char **envp)
.text:000000000000129D public main
.text:000000000000129D main proc near
.text:000000000000129D
.text:000000000000129D var_10= qword ptr -10h
.text:000000000000129D var_4= dword ptr -4
.text:000000000000129D
.text:000000000000129D ; _unwind {
.text:000000000000129D push rbp
.text:000000000000129E mov rbp, rsp
.text:00000000000012A1 sub rsp, 10h
.text:00000000000012A5 mov [rbp+var_4], edi
.text:00000000000012A8 mov [rbp+var_10], rsi
.text:00000000000012AC call _Z5Loginv ; Login(void)
.text:00000000000012E1 mov eax, 0
.text:00000000000012E6 leave
.text:00000000000012E7 retn
.text:00000000000012E7 ; } // starts at 129D
.text:00000000000012E7 main endp
.text:00000000000012E7
```

main()

The function called inside the **main** performs all the logical operations to check whether the supplied string is correct or wrong. Below is the disassembly and program workflow of how the correct or incorrect string we supply are processed.

Z5Loginv Disassembly

```
.text:000000000000129D var_10= qword ptr -10h
.text:000000000000129D ; _unwind {
.text:000000000000129D push rbp
.text:000000000000129E mov rbp, rsp
.text:00000000000012A1 sub rsp, 10h
.text:00000000000012A5 mov [rbp+var_4], edi
.text:00000000000012A8 mov [rbp+var_10], rsi
.text:00000000000012AC call _Z5Loginv ; Login(void)
.text:00000000000012E1 mov eax, 0
.text:00000000000012E6 leave
.text:00000000000012E7 retn
.text:00000000000012E7 ; } // starts at 129D
.text:00000000000012E7 main endp
.text:00000000000012E7
```

password string

wrong password block

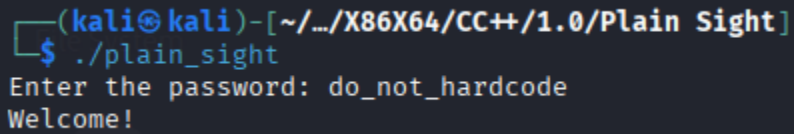
correct password block

I have intentionally highlighted the important parts of the disassembly. Check the passing string **that is basically our password**. If we supply the correct password the **correct password block** is executed and we get the **Welcome!** as stdout otherwise the **wrong password block** is executed and the program exits with the message **Wrong password!**.

Testing our input

Now lets check what happens if we supply the “**do_not_hardcode**” string as our user input.

Final Testing

A terminal window with a dark background. The prompt is (kali@kali)-[~/.../X86X64/CC++/1.0/Plain Sight]. The user enters ./plain_sight. The program prompts 'Enter the password: do_not_hardcode' and then outputs 'Welcome!'.

```
(kali@kali)-[~/.../X86X64/CC++/1.0/Plain Sight]  
$ ./plain_sight  
Enter the password: do_not_hardcode  
Welcome!
```

Yes!!! we were able to crack the binary.